

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly coupled with graph-structured data for relational learning tasks such as node classification, a setting long dominated by Graph Neural Networks (GNNs). While this coupling harnesses rich relational signals to advance predictive performance, the adversarial robustness of the resulting graph-aware LLMs has remained essentially uncharacterized. We present the first systematic study of these vulnerabilities for the learned-projector family of graph-aware LLMs, transferring established graph adversarial attacks—spanning both the poisoning (training-time) and evasion (test-time) regimes—to two representative and architecturally distinct models, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024), on text-attributed node classification. Beyond transferring existing attacks, we expose a previously unrecognized attack surface in LLAGA: by injecting malicious nodes into the placeholder positions of its node-sequence template, an adversary severely degrades classification accuracy. Our analysis traces adversarial susceptibility directly to the mechanism by which each model encodes graph structure, and it yields three concrete findings: (1) the node-sequence template of LLAGA is a decisive source of structural vulnerability; (2) the GNN encoder of GRAPH-PROMPTER confers markedly greater structural robustness; and (3) both models are acutely and consistently susceptible to imperceptible feature-perturbation attacks—a threat that the GNN literature has largely overlooked. Building on these findings, we propose GALGUARD, an end-to-end defense that unifies an LLM-based feature-correction module, which repairs feature-level perturbations, with adapted GNN defenses that counter structural attacks, and that consistently restores robustness across the evaluated datasets and attack types¹

1 Introduction

Graphs, whose nodes and edges encode entities and the relations among them, furnish a natural abstraction for the complex structures that pervade a wide range of real-world domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), purpose-built for graph-structured data, have long been the standard instruments for such settings, attaining

strong performance on tasks such as node classification and link prediction. More recently, the striking success of large language models (LLMs)—GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), trained on vast textual corpora—has catalyzed a rapidly expanding effort to marshal their capabilities for graph-related tasks. This fusion of graph data with LLMs—termed *graph-aware LLMs* (Xie et al. 2023)—has crystallized into a distinct research program in which LLMs increasingly assume responsibilities that were once the exclusive province of GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). A growing body of evidence indeed shows that graph-aware LLMs can surpass GNNs on a variety of graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), positioning them as compelling and increasingly adopted alternatives.

This rapid adoption stands in sharp contrast to our understanding of the security of graph-aware LLMs, whose adversarial vulnerabilities remain almost entirely unexamined. Established adversarial attacks on GNNs, encompassing both poisoning (training-time) and evasion (test-time) variants (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), operate in a black-box setting in which the adversary has no access to the model’s architecture or parameters and yet undermines *model integrity*, deliberately degrading predictive performance through subtle modifications of the graph structure or the node features. Crucially, whereas GNN research has concentrated almost exclusively on structural attacks, the inherently textual character of graph-aware LLMs elevates imperceptible perturbations of node features to a first-order threat—one that the existing literature has largely overlooked, and one that motivates a robustness analysis distinct from the structure-centric account inherited from GNNs.

Taken together, these three observations—the migration of graph tasks from GNNs to LLMs, the black-box character of contemporary graph adversarial attacks, and the newly consequential threat of feature-level perturbations—motivate the central question of this work: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm, and which design choices gov-*

¹Our code will be released upon acceptance.

ern their success? Answering this question rigorously demands close attention to how graphs are integrated into the LLM, because the effectiveness of graph-aware LLMs *critically depends* on that integration. Node features are readily absorbed, given the LLM’s proficiency with text, but faithfully incorporating graph structure is substantially more intricate and requires dedicated mechanisms to capture relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore organize existing graph-aware methods into two classes according to how they encode graphs for the LLM: one that renders graph data as *textual descriptions*, converting relational information into natural language, and another that employs *learned projectors* to bridge the intrinsic gap between the graph and text domains. Because textual-description methods reduce to conventional text inputs, and are therefore already compromised by the standard text-based adversarial attacks studied for LLMs, they expose no graph-specific attack surface; we accordingly direct our empirical investigation at the more challenging and less understood learned-projector methods, instantiated by two computationally efficient and representative models, LLGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024), both of which operate on a frozen pre-trained LLM.

Our analysis demonstrates that the graph-encoding design choice of a learned-projector model systematically governs the success of adversarial attacks. In particular, we uncover a new attack surface in LLGA through which an adversary severely degrades performance by injecting malicious nodes into the placeholder positions of the node-sequence template. Building on these findings, we introduce an end-to-end graph-aware LLM defense, GALGUARD, which couples an LLM-based feature corrector with adapted GNN-based structural defenses to deliver protection against both attack surfaces at once. To our knowledge, this is the first study to systematically characterize the adversarial vulnerability of learned-projector graph-aware LLMs. We summarize our contributions as follows:

- **Encoding-based taxonomy.** We organize graph-aware LLMs into a taxonomy defined by their graph-encoding strategy, which makes explicit how distinct integration mechanisms expose distinct attack surfaces and motivates our focus on the learned-projector family.
- **Attack transferability.** We establish, for the first time, that representative GNN adversarial attacks—NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019)—transfer to learned-projector graph-aware LLMs, and we quantify how severely their poisoning and evasion variants compromise these models across three text-attributed benchmarks.
- **A new attack surface.** We identify a novel attack surface in LLGA and show that injecting malicious nodes into the placeholder positions of its node-sequence template substantially degrades performance—an attack that, on denser graphs, rivals or surpasses conventional structural attacks.
- **Feature-perturbation vulnerability.** We deliver a holis-

tic robustness assessment that incorporates imperceptible perturbations of textual node features—a threat surface neglected in the GNN literature—and show that it constitutes a particularly potent attack vector against both models we study, and that combining it with structural perturbation yields a still more damaging unified attack.

- **An integrated defense.** We propose GALGUARD, an end-to-end defense that unifies LLM-based feature correction with adapted GNN-based structural defenses and consistently restores robustness against both perturbation types across the evaluated datasets and attack regimes.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Methods for encoding graphs into LLMs fall into two principal paradigms: *textual descriptions of graphs*, in which relational information is expressed as natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured representations that the LLM can consume directly. We characterize each paradigm below and derive from this dichotomy the rationale for the class of models our study targets; Figure 5 illustrates both. A full treatment appears in Appendix A.1, and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. This paradigm serializes graph data into natural language, so that the LLM processes graphs as textual descriptions, typically paired with task-specific querying instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with textual descriptions. Complementarily, Huang et al. (2023a) examined how graph structure can be injected into LLM prompts and found that LLMs tend to process graph-related prompts as contextual paragraphs—a result that underscores the decisive role of prompt design, as further explored in (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A parallel line of work employs LLMs to enrich node features and graph structures, to act as direct predictors, or to serve as feature extractors for downstream GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that couples LLMs with GNNs, and Duan et al. (2023) introduced an effective approach for enhancing textual graph learning. While these methods offer a direct route to applying LLMs to graph tasks, they characteristically fail to capture fine-grained structural intricacies—particularly on large graphs—and remain heavily contingent on prompt engineering.

Graph Representation as a Learned Projector. To surmount the limitations of textual representations, learned-projector methods transform graph data into embeddings or graph vectors that the LLM can consume directly. Representative instances include LLGA, which combines node-level templates with a linear projector (Chen et al. 2024), and approaches that integrate GNN layers into a pre-trained

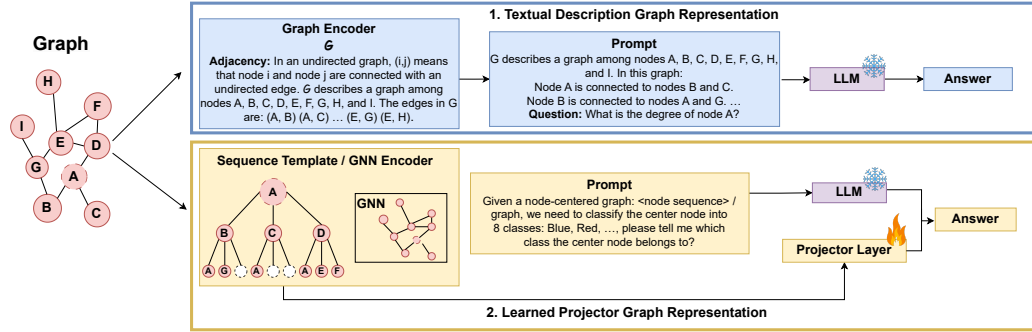


Figure 1: **Graph encoding-based adaptations of LLMs.** Under the textual-description representation, graphs are rendered in natural language (top). Under the learned-projector representation, graphs are encoded either by node templates that serialize the graph into sequences or by GNNs (bottom). In both cases the LLM remains frozen; only the projector that maps the graph into tokens is trained.

LLM (Liu et al. 2024; Qin et al. 2023). Further variants employ graph neural prompts to encode knowledge graphs (Tian et al. 2024), construct prompt-based node feature extractors coupled to a GNN adapter (Huang et al. 2023b), or condense graph information into fixed-length prefixes via graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). **This Work:** We concentrate on projector methods built on *frozen pre-trained LLMs*, which are computationally efficient and sidestep the substantial cost of retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our study centers on two representative models that together instantiate the two dominant designs within this paradigm: LLAGA (Chen et al. 2024), which pairs node-level templates with a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which pairs a GNN encoder with a linear projector. We deliberately set aside textual-description methods on two grounds: they scale poorly to large graphs, and—by reducing the graph to plain text—they expose no graph-specific attack surface of the kind our investigation targets, since they inherit precisely the text-based vulnerabilities already documented for LLMs.

3 Threat Model

We ground our threat model in established research on graph adversarial attacks and extend it to the learned-projector graph-aware LLM setting (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). Its assumptions are directly realized in practice. In social networks, an adversary typically controls only a handful of bot accounts—precisely the limited-access regime we adopt—and manipulating a small subset of nodes suffices to evade detection or to sway the classification of legitimate users. On Wikipedia, hoax articles with sparse connections to legitimate content can be strategically linked so as to induce misclassification of genuine articles’ categories. The transferability of such attacks is especially consequential given that graph-aware LLMs are strong candidates for deployment in critical applications—recommendation systems, social-media analysis, and even healthcare—where

they may supplant GNNs. Following prior work on graph adversarial attacks, we specify the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—and then state the attacker’s goal.

Model Access. The adversary operates in a strict black-box setting, with no knowledge of the model’s architecture or parameters, and influences the model only indirectly, by interacting with its training or test data.

Adversarial Capabilities. Consistent with established poisoning (evasion) attacks, the adversary is granted access to the target model’s training (inference) data. Within a pre-defined perturbation budget, and subject to unnoticeability constraints designed to avoid detection, the adversary may modify existing data or inject new data.

Attack Strategies. The adversary wields two complementary strategies: feature manipulation, which alters node features, and structure manipulation, which adds or removes edges.

Attacker’s Goal. The adversary seeks to compromise model integrity by degrading overall predictive performance—concretely, by raising the misclassification rate of the node-classification model.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016)—which span small-to-medium scales and a range of sparsity levels and which serve as the standard benchmarks for the class of models we study (Chen et al. 2023a, 2024; Liu et al. 2024). To probe scalability to large, real-world graphs, we additionally evaluate on the substantially larger ArXiv dataset (Hu et al. 2020) in a dedicated experiment (see Appendix A.5). Following (Chen et al. 2024), we adopt the standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, held fixed across all models and attacks to ensure comparability. Detailed statistics are reported in Table 1.

Tasks. We adopt *node classification* as our task throughout—the canonical benchmark for graph machine-learning models and the standard setting for evaluating graph adversarial at-

Table 1: Statistics of the evaluation datasets, which span small-to-medium scales and a range of sparsity levels, from the denser Cora to the sparser PubMed.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

tacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023)—which yields a controlled and directly comparable testbed across models, attacks, and datasets.

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels. To ensure statistical reliability, each experiment is repeated three times, and we report the mean together with its standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now justify our selection of graph-aware LLMs and adversarial attack methods, made so that our conclusions span the two dominant projector architectures and the two canonical attack formulations rather than the idiosyncrasies of any single system.

Graph-Aware LLM Methods. We study two representative learned-projector graph-aware LLMs, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), which together instantiate the two prevailing projector designs. Both build on a *frozen pre-trained LLM*, which renders them computationally efficient and readily adaptable across graph tasks and LLM backbones. LLAGA encodes graph structure through node-level templates followed by a linear projector, whereas GRAPHPROMPTER encodes it through a GNN-based encoder followed by a linear projector; in both cases the projector is a two-layer multi-layer perceptron (MLP). The foundational LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLaMA2-7B (Touvron et al. 2023), respectively; we disentangle the contribution of this backbone choice from that of the encoding mechanism in our ablation.

Adversarial Attack Methods. Our investigation employs the poisoning and evasion variants of two representative and formulation-complementary graph adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted attack that computes minimal perturbations to misclassify an individual node, whereas METAATTACK is untargeted and treats the graph structure as a hyperparameter to be optimized. Following (Li et al. 2023; Zügner and Günnemann 2019), we convert NETTACK into an untargeted evaluation by sequentially targeting each node and aggregating the results. Across all experiments, and in accordance with the unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019), we perturb at most 10% of the edges, so that structural budgets re-

main comparable to those adopted in prior work. Full details appear in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We analyze poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to each architecture, keeping the graph-encoding pipeline of each model intact so that the attack acts on the same representation the model was designed to consume. Under poisoning of LLAGA, the perturbed graph is used to regenerate the node-level sequences, and the projector is retrained from scratch on them; under poisoning of GRAPHPROMPTER, the perturbed graph is re-encoded with a GAT and the projector is likewise retrained. Under evasion, both models are trained exclusively on unperturbed data, and perturbations are introduced only at inference. Because the frozen LLM backbone is never updated, poisoning in this setting amounts specifically to retraining the projector on the perturbed graph.

Results. Table 2 yields three consistent findings for the learned-projector models under study. First, the two regimes are sharply asymmetric: poisoning exerts only a limited effect, degrading accuracy by at most 11 percentage points across both attacks, whereas evasion is markedly more damaging, inducing degradations of up to 51 points. We attribute the poisoning resilience to the models’ robust learned representations and to the stabilizing contextual signal supplied by the frozen LLM, which retraining on the perturbed graph only partially disturbs. Second, METAATTACK consistently outperforms NETTACK in both regimes and on every dataset, a gap that follows directly from METAATTACK’s treatment of graph edges as optimizable hyperparameters, which yields small yet highly effective perturbations. Third, attack effectiveness scales with graph density: denser graphs are systematically more vulnerable than sparse ones. Under evasion, for instance, LLAGA suffers a 51% accuracy decrease on the denser Cora but only a 19% decrease on the sparser PubMed, identifying structural density as a principal determinant of susceptibility.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) establish that LLAGA is consistently more susceptible to structural attacks than GRAPHPROMPTER, and they trace this disparity to LLAGA’s heightened *sensitivity to structural perturbations*. LLAGA encodes graph structure through two coupled mechanisms: a fixed-length node-level sequence generated for each node, and a graph Laplacian embedding. Because each position in this sequence corresponds uniquely to a relative structural position in the original graph, and because the node embeddings are themselves derived directly from that structure, a structural perturbation propagates immediately into the representation consumed by the LLM, producing pronounced degradation.

Table 2: Node-classification accuracy of the graph-aware LLMs LLAGA and GRAPH PROMPTER under NETTACK and METAATTACK, in both the poisoning and evasion regimes, on Cora, Citeseer, and PubMed. Each entry is the mean accuracy over three runs (standard deviation in subscript); the value in parentheses is the percentage-point drop relative to the clean baseline, so that larger drops indicate more effective attacks. Two patterns are immediate: evasion attacks are consistently more damaging than poisoning, and the largest degradations—up to 51 points—fall on LLAGA.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

GRAPH PROMPTER, by contrast, leaves its textual node embeddings intact under structural perturbation, since only the GNN component is affected; the unperturbed textual features then augment the perturbed structure and confer substantially greater robustness. This contrast isolates the encoding mechanism itself—rather than the choice of LLM backbone—as the decisive factor governing structural robustness, a conclusion our backbone ablation independently corroborates.

6 Node Sequence Template Injection Attacks

We now introduce a new attack surface in LLAGA that exploits the neighborhood detail template through which it constructs its node sequences. The attack is illustrated in Figure 2, and the corresponding algorithm is given in Appendix A.4. To obtain the node sequence for a target node u , the neighborhood detail template in LLAGA first builds a *fixed-shape* computational tree whose children at each node are its sampled 1-hop neighbors; the sampling is applied recursively to populate every subsequent level of the tree. When a node has fewer neighbors than required, placeholders are inserted to preserve the fixed fan-out of, *e.g.*, $k = 10$ children, in accordance with LLAGA’s original design (see the “Clean” case in Figure 2 and lines 1–14 of the algorithm). Because the adversary operates through ordinary graph manipulation—the same edge-modification capability assumed by the structural attacks above—rather than by editing the template directly, it is precisely this reliance on placeholders to maintain a fixed structure that opens the exploitable attack surface: edges added to u deterministically dictate which nodes occupy those placeholder slots.

Exploiting this neighborhood detail template, an adversary injects malicious nodes into the placeholder positions via graph manipulation and thereby degrades performance substantially. We formalize three injection strategies of monotonically increasing potency. *Non-Adjacent Injection (NI)* inserts nodes drawn from outside the 2-hop neighborhood of the target node u (see the “Attacked” case in Figure 2 and lines 21–27). *Supernode Injection (SI)* instead injects a single high-degree node into a placeholder, again constrained to lie outside the target’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* generalizes SI by distributing several distinct supernodes across the available placeholders (lines 35–43). A

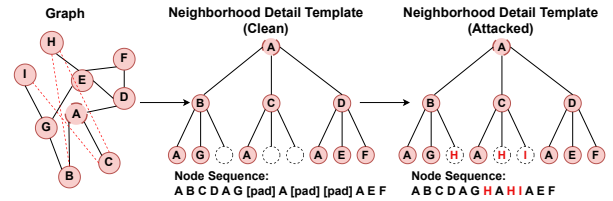


Figure 2: Illustration of the node sequence template injection attacks. The depicted instance is the Non-adjacent Injection (NI) attack, in which red dashed lines denote the adversary’s edge perturbations. For the Supernode Injection (SI) and Multiple Supernode Injection (MSI) attacks, one or several high-degree supernodes are injected into the placeholder positions. Here, $k = 3$.

single mechanism underlies all three: introducing uninformative nodes into the node sequence corrupts the target node’s representation, which in turn yields a poorly trained projection layer and compromises the integrity of both learning and inference.

Results. As reported in Table 3, LLAGA remains highly resilient to the injection attacks under poisoning on Cora and Citeseer, yet degrades sharply under evasion: accuracy falls to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the *NI*, *SI*, and *MSI* attacks, respectively. The strictly monotone degradation from *NI* through *SI* to *MSI* confirms that injecting higher-degree and more numerous supernodes produces progressively stronger attacks, exactly as the shared corruption mechanism predicts.

On the larger PubMed graph, by contrast, the model is considerably more resilient to every injection strategy, incurring only a modest evasion drop (to 0.85, 0.82, and 0.76). This pattern admits a natural locality explanation: in the smaller, tightly connected Cora and Citeseer graphs a localized injection exerts a concentrated effect that severely disrupts the model’s reasoning over the local neighborhood, whereas in the far larger PubMed graph the same perturbation is diluted and its impact attenuated.

Crucially, on the denser datasets the injection attacks are more effective than the traditional graph adversarial methods.

Table 3: Accuracy of LLAGA under the node-sequence template injection attacks across datasets and attack regimes. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively, and *Clean* denotes performance without any attack. Evasion is uniformly far more damaging than poisoning, and attack strength intensifies monotonically from NI through SI to MSI; the effect is pronounced on Cora and Citeseer and attenuated on the larger PubMed graph.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

Under evasion, *MSI* drives accuracy down to 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas *METAATTACK*—the strongest structural baseline—reaches only 0.44, 0.50, and 0.73 (Table 2). On Cora and Citeseer, this establishes the node-sequence template as a more damaging attack surface than the standard structural perturbations previously studied for GNNs, and thereby locates a vulnerability specific to LLAGA’s serialization mechanism.

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks concentrate almost exclusively on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); the sole feature-oriented exception, *NETTACK* (Zügner, Akbarnejad, and Günnemann 2018), does not transfer to the textual domain. Since structural attacks are only moderately effective against the more robust *GRAPH-PROMPTER*, and since our earlier analysis identifies textual features as central to model robustness, we now examine *visually imperceptible* feature-perturbation attacks as a direct route to degrading performance.

We adapt the imperceptible attack strategies of (Boucher et al. 2022) with one key modification: perturbations are generated against a task-agnostic translation objective (English to French). The objective is to alter the input text so that its rendered form remains visually indistinguishable to a human reader while inducing an incorrect translation, on the guiding intuition that a corrupted translation reflects corrupted internal semantics. We instantiate two such attacks: *Homoglyphs*, which substitute characters with visually indistinguishable counterparts, and *Reorderings*, which employ directional control characters to alter the underlying character sequence without changing the rendered text. Because both attacks operate at the level of the raw byte sequence rather than the glyphs a reader perceives, their imperceptibility is

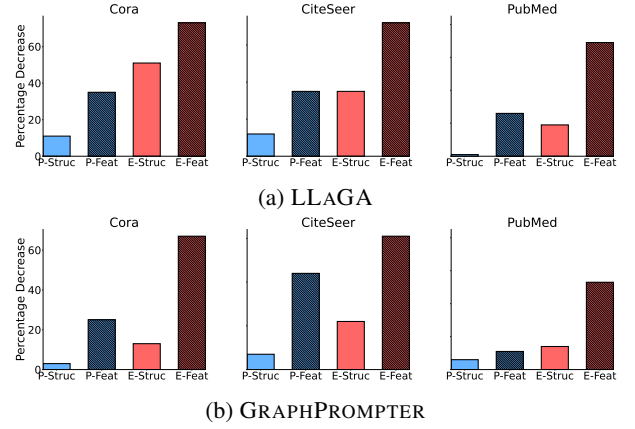


Figure 3: Comparison of structure-based and feature-based adversarial attacks on the two graph-aware LLMs. Attack success is measured as the percentage decrease in accuracy relative to the clean baseline (y-axis). P-Struc, P-Feat, E-Struc, and E-Feat denote the poisoning and evasion regimes under structural and feature perturbations, respectively. For both models, feature-based attacks dominate structure-based attacks in every regime, reorienting the vulnerability profile away from that of traditional GNNs.

a property of the rendering; representative examples of both appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations via differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model’s outputs on the perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature, so that feature and structural budgets are held to comparable, tightly bounded fractions of the input.

Results. As reported in Table 4, Reordering attacks are uniformly more potent than Homoglyph attacks across every model and dataset. On Cora and CiteSeer, LLAGA is vulnerable to both: under poisoning, accuracy falls by 27% and 23% under Homoglyph and by a larger 35% and 25% under Reordering, respectively. In every configuration *GRAPH-PROMPTER* is the more resilient of the two models, and PubMed follows the same qualitative trends with a somewhat attenuated overall impact relative to the smaller datasets.

A direct comparison against structural perturbations, presented in Figure 3, reveals a fundamental reorientation of the vulnerability landscape for these models. Feature-perturbation attacks—Reordering in particular—outperform the strongest structural attack (*METAATTACK*) by a wide margin on both LLAGA and *GRAPH-PROMPTER*. This establishes that both learned-projector graph-aware LLMs weight input features more heavily than graph structure and are correspondingly more exposed to feature-level perturbation—a decisive departure from the structure-dominated vulnerability profile of traditional GNNs. We probe this phenomenon further in the next section by combining the two attack types into a unified setting.

Table 4: Accuracy of LLAGA and GRAPHPROMPTER under the visually imperceptible feature-perturbation attacks (Homoglyph and Reordering), in both the poisoning and evasion regimes; the value in parentheses is the percentage-point drop from clean accuracy. Reordering is uniformly the stronger attack, evasion far exceeds poisoning, and GRAPHPROMPTER is the more resilient model in every configuration.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 ± 0.05 (27%)	0.52 ± 0.04 (13%)	0.41 ± 0.04 (54%)	0.38 ± 0.03 (37%)
	Reordering	0.58 ± 0.06 (35%)	0.45 ± 0.05 (25%)	0.24 ± 0.05 (73%)	0.20 ± 0.03 (67%)
Citeseer	Homoglyph	0.54 ± 0.04 (23%)	0.65 ± 0.04 (7%)	0.32 ± 0.03 (55%)	0.52 ± 0.05 (26%)
	Reordering	0.46 ± 0.03 (35%)	0.39 ± 0.02 (44%)	0.20 ± 0.05 (72%)	0.27 ± 0.02 (61%)
PubMed	Homoglyph	0.72 ± 0.04 (20%)	0.83 ± 0.04 (8%)	0.36 ± 0.06 (60%)	0.55 ± 0.03 (39%)
	Reordering	0.67 ± 0.05 (26%)	0.80 ± 0.03 (11%)	0.28 ± 0.04 (69%)	0.42 ± 0.04 (53%)

8 A Unified Imperceptible Attack on Graph-aware LLMs

We now propose a unified attack that jointly applies feature and structural perturbations, forming a maximally imperceptible yet highly effective adversarial strategy. Our hypothesis is that combining the two perturbation types degrades accuracy far more severely than either alone while keeping the alterations to both the graph structure and the node features within their respective unnoticeability budgets.

The structural component uses METAATTACK, which subtly rewires edges, and the feature component combines the Homoglyph and Reordering attacks introduced in the previous section. Acting in concert under the same 10% budgets adopted throughout, these perturbations constitute a highly inconspicuous attack that simultaneously targets the relational and the textual information on which the two graph-aware LLMs jointly depend.

Results. Table 5 confirms this hypothesis: across all datasets the unified attack is substantially more effective than either perturbation method in isolation, reducing accuracy by as much as 87% under evasion. This result crystallizes a central architectural insight into these graph-aware LLMs: unlike traditional GNNs, whose vulnerability is dominated by structural perturbation, both models are acutely susceptible to feature-level attacks and, a fortiori, to their combination. The efficacy of the unified attack demonstrates that they depend on the mutual consistency of graph structure and node features, which renders them highly vulnerable to coordinated, inconspicuous modifications of both at once.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize the relationship between perturbation intensity and attack success, we vary the fraction of perturbed features from 1% to 10%. Our analysis of the Reordering attack, whose behavior mirrors that of the Homoglyph attack, reveals a consistent monotone trend across both models and all three datasets (Cora, Citeseer, and PubMed): reducing the fraction of perturbed features predictably weakens the attack.

Table 5: Accuracy under the unified imperceptible attack, which jointly applies structural (METAATTACK) and feature (Homoglyph + Reordering) perturbations; the value in parentheses is the percentage-point drop from clean accuracy. The combined attack is markedly more destructive than either component in isolation, reducing accuracy by as much as 87% under evasion.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 ± 0.05 (61%)	0.28 ± 0.08 (53%)
	Evasion	0.14 ± 0.06 (84%)	0.11 ± 0.05 (82%)
Citeseer	Poisoning	0.26 ± 0.04 (63%)	0.22 ± 0.03 (69%)
	Evasion	0.10 ± 0.09 (86%)	0.13 ± 0.07 (81%)
PubMed	Poisoning	0.39 ± 0.07 (57%)	0.32 ± 0.04 (64%)
	Evasion	0.12 ± 0.05 (87%)	0.19 ± 0.04 (79%)

Notably, as the perturbation level decreases, the success rates of the poisoning and evasion variants converge. This convergence follows directly from the diminishing adversarial signal: at low perturbation levels the injected signal is too weak to materially affect the model, regardless of whether it is introduced during training (poisoning) or at inference (evasion). A detailed plot of these trends is provided in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We next assess whether the choice of foundation LLM influences the adversarial robustness of these graph-aware LLMs. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). To isolate the effect of the backbone, we replace LLAGA’s LLM with Llama2-7B and compare performance under poisoning and evasion attacks across all datasets. The two backbones attain essentially identical clean accuracy (*e.g.*, Vicuna: 0.89 ± 0.07 ; Llama2: 0.90 ± 0.03 on Cora) and exhibit comparable degradation under attack, with all differences well within one standard deviation; the same behavior holds when the backbone is swapped in GRAPHPROMPTER. These results establish that, within the regimes we evaluate, adversarial robustness is governed by the graph-encoding mechanism rather than by the choice of foundation LLM. Full results are reported in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses are ill-suited to graph-aware LLMs precisely because of the dual vulnerability—to both structural and feature perturbations—that our attacks expose. Many such defenses reweight or prune edges on the basis of node-feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024) and therefore presuppose access to clean features; once the features are themselves perturbed, these methods fail outright (see Figure 3 in the Appendix). Defenses that decouple structure from features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu

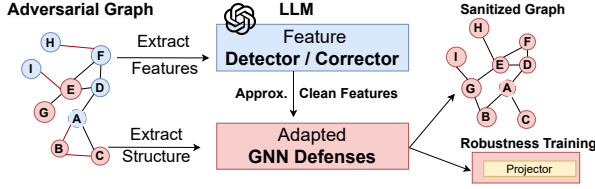


Figure 4: **Overview of the GALGUARD defense.** Perturbed node features and edges are shown in blue and red, respectively. An LLM first detects perturbations in the node features extracted from the adversarial graph and emits a corrected version; the refined features, together with the perturbed edges, are then processed by the adapted GNN defenses, which either yield a sanitized graph or are folded into the projector’s robustness training. The two stages target the feature and structural attack surfaces in turn.

et al. 2021), and even OCR-based text sanitization (Boucher et al. 2022), likewise collapse under a coordinated attack that perturbs both surfaces at once. This dual exposure renders a new, integrated defense not merely desirable but necessary.

We therefore propose the graph-aware LLM defense (GALGUARD), an end-to-end strategy that couples two complementary components—(1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses—which address the feature and structural attack surfaces, respectively.

Feature Robustness Strategy. As illustrated in Figure 4, an LLM acts as an adaptive analyzer that inspects the textual node features for anomalous patterns and issues approximate corrections that restore feature integrity, thereby neutralizing feature-level perturbations. Because it operates directly on the text and requires no retraining of the target model, this LLM-based correction constitutes a practical and readily deployable remedy for feature vulnerabilities. The specific instructions and prompt are provided in Appendix A.6; across all experiments we instantiate the corrector with GPT-4 Turbo.

Graph Structure Robustness Strategy. To reinforce structural robustness, we adopt a two-step procedure. First, as a preprocessing step, we apply a *graph purification heuristic* (Wu et al. 2019): for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and every neighbor embedding $\phi(x_u)$ and remove those edges whose similarity falls below a threshold. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPH PROMPTER, GNNGuard’s principles apply directly to its message-passing layers. For LLAGA, whose architecture lacks explicit message passing, we devise a tailored adaptation: we fold the purification step into the construction of the computational tree that generates the node sequence, and we introduce a learnable *global structural context embedding* (M_{global}) as an additional parameter of LLAGA’s projector. Concatenating M_{global} with the purified node-sequence embedding and training the two jointly teaches the model to operate robustly on potentially perturbed graph inputs. The design of GALGUARD is deliberately modular and plug-and-play, so that

alternative structural defenses such as Pro-GNN (Jin et al. 2020) can be substituted for GNNGuard without further modification.

10.1 Results

Table 6 reports the accuracy of both models under METAATTACK alongside two variants of our defense: GALGUARD_p, which combines the feature corrector with the graph-purification module, and the full GALGUARD, which additionally integrates GNNGuard (or its LLAGA adaptation). Across every evaluated dataset (Cora, CiteSeer, and PubMed) and in both the poisoning and evasion regimes, our defenses consistently and substantially mitigate the impact of METAATTACK. GALGUARD_p already furnishes an effective foundation, confirming that jointly correcting feature perturbations with the LLM corrector and purifying the graph structure yields marked robustness gains. The full GALGUARD improves upon GALGUARD_p in every setting—most strikingly recovering LLAGA’s Cora evasion accuracy from 0.44 to 0.83—which demonstrates the synergistic benefit of layering the GNN-based structural defense atop feature correction.

Table 6: Accuracy of LLAGA and GRAPH PROMPTER under METAATTACK and the proposed defense GALGUARD, in both the poisoning and evasion regimes. GALGUARD_p combines the LLM feature corrector with graph purification, and the full GALGUARD additionally integrates GNNGuard (or its LLAGA adaptation). The best result per column is in bold; the full GALGUARD attains the strongest robustness in every setting.

Data	Method	LLAGA		GRAPH PROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper presents the first systematic characterization of the adversarial vulnerabilities of learned-projector graph-aware LLMs, a previously unexplored yet increasingly consequential problem. Our central finding is that the mechanism by which graph data is integrated into the LLM is itself a critical source of vulnerability: we uncover new attack surfaces and show that design choices such as LLAGA’s node-sequence template markedly amplify a model’s susceptibility. We further establish that feature-perturbation attacks are highly effective—in sharp contrast to their negligible effect on traditional GNNs—frequently surpassing structural attacks, and

that a unified attack combining both perturbation types can render these models near-useless.

To counter these threats, we propose GALGUARD, an end-to-end defense that unifies an LLM-based feature corrector with adapted GNN-based structural defenses and consistently restores robustness across the evaluated datasets and attack types. As graph-aware LLMs are increasingly adopted in high-stakes domains such as social networks, healthcare, and finance, guaranteeing their resilience to adversarial exploitation becomes essential. We intend this work as a foundation for that effort—one that both delineates the vulnerabilities of learned-projector graph-aware LLMs and supplies a concrete, extensible defense to safeguard their deployment, and that charts a clear path toward extending this analysis to further encoding paradigms and graph tasks.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.

- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 develops a detailed taxonomy of graph-aware LLMs, and Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 details the adversarial attacks NETTACK and METAATTACK, and Appendix A.4 specifies the algorithm underlying the node-sequence injection attack on LLAGA. Appendix A.5 reports our ablation studies—the effect of varying the fraction of perturbed features and the effect of the base LLM on attack performance. Finally, Appendix A.7 provides concrete examples of the feature attacks together with the LLM-corrector template, and Appendix A.8 discusses the broader implications of our findings.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has rapidly emerged as an active research frontier, with a diverse body of work exploring how LLMs can be adapted or augmented for graph-related tasks such as reasoning, representation, and inference. These approaches partition naturally into two categories according to how they encode graphs for the LLM: *textual descriptions of graphs*, in which relational information is expressed as natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured representations that the LLM can process directly. This section surveys the methods within each encoding paradigm and, in doing so, delineates the design space from which we draw the two models our study targets. Figure 5 illustrates both approaches, and Table 7 summarizes their principal distinctions.

Graph Representation as a Textual Description In this first category, graph data is expressed as natural language, so that the LLM processes graph structures as textual descriptions. This paradigm has been explored extensively, with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, typically paired with task-specific querying instructions. For instance, Tan et al. (2023) proposed a unified framework for attributed graph embedding that generates random-walk-based textual sequences and uses them to fine-tune LLMs. Similarly, Zhao et al. (2023) addressed graph reasoning within text space, translating graph structures into natural language to facilitate LLM interaction. Extending this line, Wei et al. (2024a) augmented textual graph representations with visual ones, strengthening the reasoning capabilities of LLMs on graph data. In multimodal settings, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) examined how injecting graph structure into LLM prompts affects predictive performance and found that LLMs tend to process graph-related prompts as contextual paragraphs rather than

explicitly recognizing graph structure—a result that underscores the decisive role of prompt design, as also evidenced by (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extend LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) employed LLM-generated explanations as features to improve both the performance and the interpretability of LLMs on graphs.

While still representing graphs as textual descriptions, a further line of work couples GNN-based pretraining strategies with LLMs to strengthen the latter’s graph-reasoning capabilities. For instance, Chen et al. (2023a) proposed two LLM–GNN integration strategies: (1) using LLMs to enrich node features for node classification, and (2) employing LLMs as direct predictors for graph tasks. In a similar spirit, Chandra et al. (2020) combined LLMs as text encoders with GNNs as graph-structure encoders for tasks such as fake-news detection in online communities. Chen et al. (2023b) studied label-free node classification, using LLMs to annotate nodes whose labels then supervise a GNN, while Mavromatis et al. (2023) employed an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that couples LLMs and GNNs through a tunable side structure attached to each LLM layer, improving both training efficiency and inference speed. Complementarily, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that enhances textual graph learning with LLMs: the LLM is first fine-tuned on a downstream task such as node classification, and node embeddings are subsequently obtained from the last hidden states of the fine-tuned model. Additionally, Zhang (2023) proposed Graph-Toolformer, which leverages ChatGPT-driven prompt engineering to augment the graph-reasoning capabilities of LLMs.

Although these approaches broaden the accessibility of LLMs for graph-related tasks, they characteristically fail to capture the fine-grained structural intricacies inherent in graph data. Moreover, their performance remains heavily contingent on effective prompt engineering—a demanding process that requires careful design and repeated experimentation to approach optimal results.

Graph Representation as a Learned Projector To overcome the challenges of textual-representation approaches, learned-projector methods transform graph data into embeddings or graph vectors that the LLM can process more directly. Chen et al. (2024), for example, introduced LLAGA, a large language and graph assistant that preserves the general-purpose character of the LLM while converting graph structures into graph tokens amenable to LLM processing. Along similar lines, (Liu et al. 2024; Qin et al. 2023) injected graph-structure information into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) employed graph neural prompts to encode knowledge graphs, transforming them into graph embeddings that the LLM can leverage at inference time. Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs, combining a graph

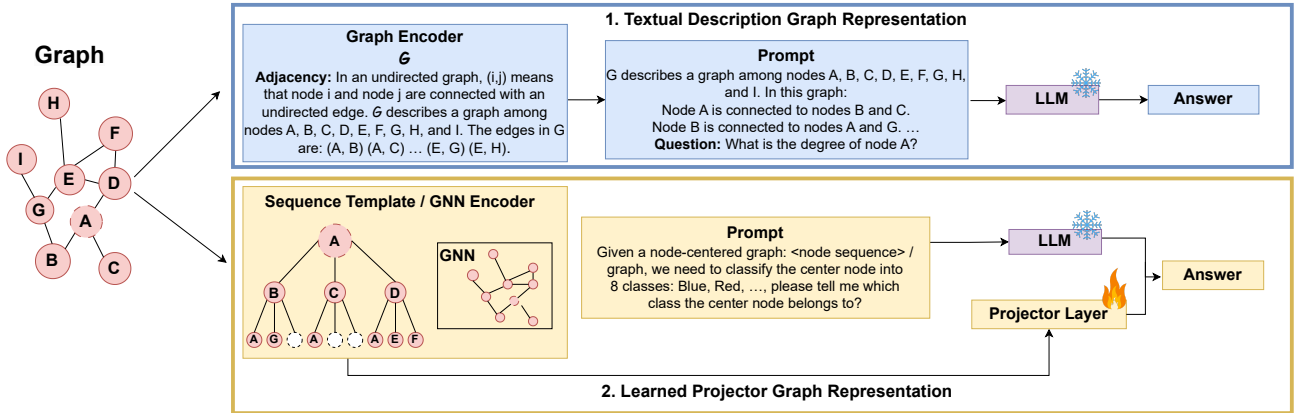


Figure 5: **Graph encoding-based adaptations of LLMs.** Under the textual-description representation, graphs are rendered in natural language (top). Under the learned-projector representation, graphs are encoded either by node templates that serialize the graph into sequences or by GNNs (bottom). In both cases the LLM remains frozen; only the projector that maps the graph into tokens is trained.

Table 7: Comparison of the two graph-encoding paradigms for adapting LLMs, contrasting the textual-description and learned-projector approaches along six axes: graph encoding, LLM modification, structural representation, flexibility, computational cost, and suitability for graph tasks.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

adapter (a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

A further promising direction condenses graph information into fixed-length prefixes via graph transformers attached to each LLM layer to enhance its reasoning (Chai et al. 2023). Yang et al. (2021) developed GraphFormers, which interleaves text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model, and Peng et al. (2024) developed Chat-Graph, an interactive interface that lets users query their graphs in natural language by representing graphs as sequential paths and fine-tuning the LLM on the graph data to support analysis.

We deliberately exclude projector adaptations that require retraining the LLM, since they incur substantial computational cost without commensurate performance gains. We concentrate instead on adaptations built on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This design not only reduces computational cost but also affords greater flexibility in integrating diverse LLM backbones, allowing us to exploit the extensive knowledge and capabilities of LLMs—without the overhead of retraining—while still adapting them to specific graph tasks. Throughout this work we adopt two representative projector methods that together span the two dominant designs: **LLAGA** (Chen et al. 2024), which pairs node-level templates with a linear projector, and **GRAPH-PROMPTER** (Liu et al. 2024), which employs a GNN-based projector to encode the graph.

This work. We do not empirically evaluate textual-description methods, which characteristically fail to capture the structural intricacies inherent in graph data. Such methods are generally confined to simpler tasks—for example, describing basic graph properties such as node degree—and become infeasible on larger graphs for reasons of scalability: the prompt token limit sharply constrains the detailed descriptions that larger structures demand, and the required descriptions quickly exceed the allowable token budget. From a vulnerability standpoint, moreover, textual-description methods expose no unique attack surface: because the graph is rendered as plain text (e.g., “Node A is connected to Node B”), they are compromised by conventional text-based adversarial attacks on LLMs rather than by the structural or feature perturbations specific to graph-based models. They therefore lack the graph-specific structure that gives rise to the novel vulnerabilities central to our investigation, which is precisely why our empirical study concentrates on the learned-projector family.

A.2 Attacks on GNN

GNNs are exposed to a broad spectrum of attacks: membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a specific node belonged to the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s outputs and learned embeddings to infer sensitive or missing node attributes; property inference attacks (Zhang

et al. 2022; Wang and Wang 2022), which extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade predictive performance through subtle perturbation of the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which recover the underlying graph structure from the learned embeddings or model outputs.

Among these, adversarial attacks are of particular concern owing to their direct impact on predictive performance. Such attacks introduce small, carefully crafted *perturbations*—modifying node features or rewiring edges—so as to substantially degrade the GNN’s accuracy. They fall into two categories: **poisoning attacks**, mounted during the *training phase* by manipulating the training data, and **evasion attacks**, mounted at *test time*, in which the adversary perturbs the input graph to mislead the model’s predictions. Zügner, Akbarnejad, and Günnemann (2018), for instance, demonstrated that adversarial perturbation of graph structure or node features can markedly degrade GNN performance; their method, **NETTACK**, uses a greedy optimization technique to compute the minimal perturbation that maximizes the model’s training loss, thereby isolating the smallest change to the features or structure sufficient to disrupt the model’s predictions. Li et al. (2023) revisited graph adversarial attacks from a data-distribution perspective, showing how adversarial examples can significantly shift the underlying data distribution and proposing a defense to counteract this effect. In a similar vein, Wu et al. (2019) provided a comprehensive analysis of adversarial examples on graph data that yields deep insight into both attacks and defenses; focusing on the sensitivity of GNNs to small perturbations, they proposed defense techniques that detect and counteract adversarial modifications and thereby enhance robustness. **METAATTACK** (Zügner and Günnemann 2019) mounts both poisoning and evasion attacks on GNNs by treating the graph structure as an optimizable hyperparameter, modifying it directly via gradient descent and formulating the attack as a bi-level optimization problem. The method exploits meta-gradients—which capture how small structural perturbations affect the attacker’s post-training loss—to compute precise structural updates that degrade model performance.

In this paper we adopt **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attacks that jointly span the poisoning and evasion strategies: poisoning attacks compromise model integrity during training, whereas evasion attacks degrade performance at inference (test) time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs the graph structure by adding and removing edges while preserving the perturbed graph’s degree distribution. It employs a greedy optimization procedure to compute the minimal perturbation that maximizes the training loss of a surrogate model—a linearized two-layer graph convolutional network (GCN) that approximates how structural changes affect the classification outcome. The objective of **NETTACK** is to increase the model’s confidence in incor-

rect predictions by maximizing the discrepancy between the logits of the perturbed and clean graphs. To this end, NETTACK enumerates candidate edges around a target node and evaluates, for each incident edge, a score quantifying the effect of altering that edge on the target node’s classification; it then greedily selects the highest-scoring edge and updates the graph by adding or removing it. Because NETTACK is a targeted attack that misclassifies a single specified node, we follow (Zügner and Günnemann 2019) in converting it into an untargeted attack that compromises the model’s overall node-classification performance: we repeatedly select a test node at random, attack it with NETTACK while accounting for all nodes in the network, and aggregate the results over the test set.

METAATTACK optimizes the graph structure directly by gradient descent, treating the adjacency matrix as a hyperparameter. It casts the attack as a bi-level optimization problem in which the inner level minimizes the training loss on the original graph and the outer level minimizes the loss on the poisoned graph, relying on a linearized two-layer GCN surrogate to approximate how structural changes affect performance. A score function then evaluates the impact of each candidate perturbation, using the sign of the meta-gradients (the gradients with respect to the hyperparameters) of connected nodes to indicate which edges to modify, and the algorithm greedily adds or removes the highest-scoring edges to maximize attack impact.

Both methods enforce the same unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018), which guarantees that the graph’s degree distribution changes only slightly; accordingly, we perturb at most 10% of the edges throughout.

For poisoning attacks on LLAGA, we take the perturbed graph structure produced by NETTACK or METAATTACK, regenerate the node-level sequences from it, and retrain from scratch the projector that maps these sequences into token embeddings. For GRAPHPROMPTER, we analogously re-encode the perturbed graph with a graph attention network (GAT) and retrain its projector. For evasion attacks, the graph-aware LLM is trained on unperturbed data, and perturbations are introduced only at the inference stage; the frozen LLM backbone remains fixed in every case.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 formalizes the node-sequence template injection attack on LLAGA. The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14); when a node has fewer than k neighbors, placeholders are inserted, and precisely these placeholders constitute the exploitable attack surface. We exploit it by injecting malicious nodes into the placeholder positions through graph manipulation, via three strategies of increasing potency: *Non-Adjacent Injection (NI)* inserts nodes drawn from outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a single high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). In every case

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

the injected, uninformative nodes corrupt the target node’s sequence representation, which in turn impairs training of the projection layer and degrades classification performance.

A.5 Ablation Studies

This section reports the full results for the ablation studies summarized in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports the performance of the two graph-aware LLMs, LLAGA and GRAPHPROMPTER, as the fraction of perturbed features is varied. For both models, increasing the proportion of perturbed features consistently and monotonically degrades accuracy, corroborating their pronounced sensitivity to feature-level attacks and confirming that the effect is governed by perturbation intensity rather than by an artifact of any single budget.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 establishes that, within the LLAGA framework, the choice of base LLM has negligible impact on both clean performance and adversarial vulnerability. Across Cora, Citeseer, and PubMed, the Vicuna-7B and Llama2-7B variants achieve nearly identical clean accuracy and exhibit closely matched degradation under both poisoning and evasion attacks. Under METAATTACK evasion on Cora, for example, accuracy falls to 0.44 ± 0.06 for Vicuna-7B and to 0.43 ± 0.08 for Llama2-7B—a difference well within one standard deviation. The same pattern holds uniformly across every dataset and attack type, demonstrating that, for the backbones evaluated, attack effectiveness is essentially independent of the base LLM and is instead governed by the graph-encoding mechanism.

A.6 Results on the ArXiv Dataset

We now evaluate our attack on the large-scale ArXiv dataset—169,343 nodes and 1,166,243 edges at a sparsity of 0.81—to gauge the effectiveness of the node-sequence template attacks on large, real-world graphs. As reported in Table 9, ArXiv exhibits a pronounced gap between the two regimes: across all three injection strategies, poisoning consistently outperforms evasion—a reversal of the pattern observed on Cora, Citeseer, and PubMed. This indicates that ArXiv possesses structural characteristics that render it more susceptible to poisoning yet more robust to evasion, and it underscores that the poisoning–evasion ordering is itself dataset-dependent. Notably, despite the graph’s very low sparsity, inserting even a single supernode suffices to degrade performance substantially.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 illustrate our adversarial attacks on real node features drawn from the Cora dataset. The “input” denotes the text supplied to the LLM after the attack has been applied. Crucially, although a human reader perceives the original and perturbed texts as visually identical or nearly so, the LLM ingests a distinct sequence of Unicode characters or a reordered character stream, which can alter its behavior—a concrete demonstration that the attacks’ imperceptibility resides in the rendered glyphs rather than the underlying bytes. We illustrate this discrepancy in Table 10 for the homoglyph attack and in Table 11 for the reordering attack.

Feature Corrector Defense Template We designed a feature-corrector module to counter the visually imperceptible attacks studied in this work—specifically, homoglyph substitutions and character reorderings. The module is realized as a prompt-based defense driven by an LLM, whose prompt is engineered to cast the LLM as a linguistic validator charged with detecting and repairing subtle, human-imperceptible perturbations.

The prompt template is shown below. By explicitly specifying the task and prescribing a clear output format, it substantially improves the LLM’s corrective performance.

A.8 Detailed Discussion

Our findings yield several critical insights into the vulnerabilities and design considerations of learned-projector graph-aware LLMs. As the integration of graph-structured data into LLMs continues to gain momentum, it is essential to examine not only the performance benefits of these architectures but also their attendant vulnerabilities. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs—LLAGA and GRAPHPROMPTER—exposes significant weaknesses, most acutely under the poisoning and evasion attacks traditionally used to undermine GNNs. We elaborate our key observations below.

Impact of Graph Encoding on Adversarial Vulnerability

The central observation from our results is that the mechanism used to integrate graph structure into an LLM directly modulates its susceptibility to adversarial attack. In particular, LLAGA’s node-sequence template creates an exploitable vulnerability that adversaries leverage to substantially degrade performance. This attack surface—unique to LLAGA among the models we study—exposes a fundamental tradeoff in learned-projector design: node-level templates furnish an effective means of encoding graph structure and yield strong task performance, yet they simultaneously introduce attack vectors absent from traditional GNNs. By contrast, GRAPHPROMPTER, which encodes graphs through a GNN-based projector, exhibits markedly greater resilience to these attacks, underscoring the advantages of hybrid architectures that fuse the complementary strengths of LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations

Beyond the vulnerabilities rooted in the graph-encoding technique, our results establish the striking effectiveness of feature-perturbation attacks against both graph-aware LLMs. Despite their divergent structural encodings, LLAGA and GRAPHPROMPTER alike display pronounced sensitivity to visually imperceptible perturbations of the node features. This reflects a broader tension in the LLM paradigm: although these models excel at processing text, their robustness to adversarial perturbation—especially perturbation targeting node features—remains a critical open challenge. That an adversary can degrade performance by subtly and undetectably altering node attributes calls the integrity of graph-aware LLMs into question, most acutely in security-sensitive applications. While text-sanitization techniques such as those of (Boucher et al. 2022) offer partial remedies, defending against imperceptible manipulations such as homoglyph substitution remains difficult. Our proposed LLM-based correc-

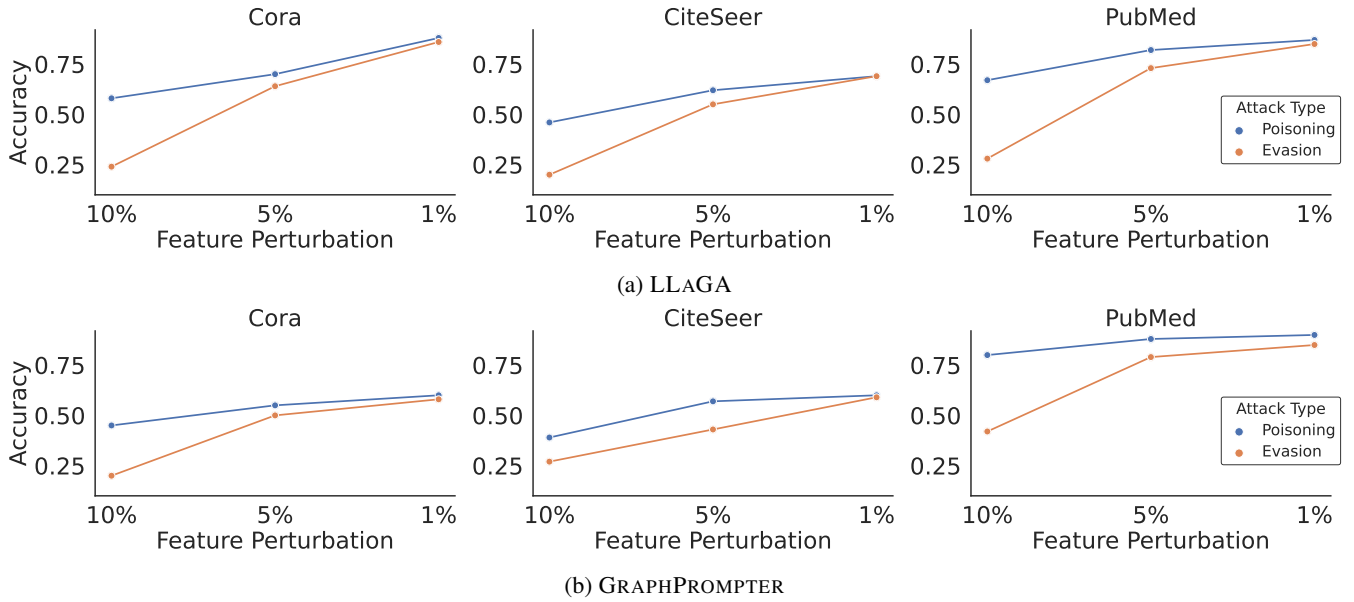


Figure 6: Performance of the two graph-aware LLMs as the fraction of perturbed features is varied. Lower accuracy indicates a more successful attack; for both models accuracy degrades monotonically with the perturbation fraction, and the poisoning and evasion variants converge as the fraction shrinks.

Table 8: Impact of the base LLM on clean performance and on vulnerability to the various attacks, shown for LLaGA with Vicuna-7B and Llama2-7B backbones. The choice of base model does not materially affect susceptibility; every difference lies within one standard deviation.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

tor addresses precisely this vulnerability by prompting the model to detect and repair suspicious or malformed inputs before downstream processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings reveal that the efficacy of poisoning and evasion attacks on graph-aware LLMs is not uniform across datasets. Whereas the attacks are especially effective on Cora, Citeseer, and PubMed, the ArXiv dataset exhibits a more nuanced interaction between attack strategy and graph structure, in which poisoning uniquely dominates evasion. This variation highlights the decisive role of dataset characteristics in assessing the vulnerabilities of graph-aware LLMs, and it cautions against extrapolating a single poisoning–evasion ordering across graphs. As these models are deployed across increasingly diverse real-world settings, their robustness must be evaluated not only with respect to algorithmic design but also in relation to

the specific domains and data on which they operate.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study carry broad implications for the future development and deployment of graph-aware LLMs. Given their accelerating adoption for tasks traditionally reserved for GNNs, it is essential to weigh the tradeoff between performance and vulnerability when designing these paradigms. As graph-aware LLMs continue to mature, there is a pressing need for more robust encoding techniques and for defenses against adversarial manipulation—most urgently against feature perturbation. Moreover, our characterization of how adversarial attacks transfer from GNNs to learned-projector graph-aware LLMs opens new avenues for research into protecting these models against sophisticated adversaries, and it provides a template for extending the same analysis to further encoding paradigms.

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgpprir heuristic fr discvring prtin squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed.text.here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature-corrector module. It explicitly specifies the LLM's role, the correction task, and the expected output format, which together sharpen its corrective behavior.

Table 9: Accuracy under the node-sequence template injection attack across datasets, including the large-scale ArXiv graph. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively, and *Clean* denotes performance without any attack. On ArXiv, poisoning uniquely outperforms evasion, reversing the ordering observed on the smaller datasets and highlighting the dataset-dependence of the two regimes.

Data	Attack	NI	SI	MSI
Cora	Clean	——	0.89 $\pm$ 0.07	——
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	——	0.71 $\pm$ 0.04	——
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	——	0.90 $\pm$ 0.04	——
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03
ArXiv	Clean	——	0.73 $\pm$ 0.02	——
	Poisoning	0.50 $\pm$ 0.03	0.43 $\pm$ 0.04	0.42 $\pm$ 0.03
	Evasion	0.71 $\pm$ 0.06	0.69 $\pm$ 0.02	0.60 $\pm$ 0.03

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have concentrated overwhelmingly on structural perturbation, since feature-based perturbations typically fail to alter the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and are therefore ineffective. Consequently, and precisely because of this limited effectiveness, there exist *no known defenses against feature-based attacks on GNNs*. As we demonstrate in the main paper, however, feature-perturbation attacks are highly effective in the graph-aware LLM paradigm. This establishes a qualitatively new challenge: feature-based attacks that were inconsequential against GNNs now successfully manipulate model behavior, exposing a defense gap that no prior GNN work addresses and rendering the development of defenses that cover both structural and feature-based perturbation imperative. While text-sanitization techniques—such as optical character recognition (OCR) applied as a preprocessing step before the text is encoded for the LLM—offer a partial remedy, defending against imperceptible manipulations such as homoglyph substitution remains difficult, since OCR is imperfect and misinterprets homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). These considerations directly motivated our design of an LLM-based self-correcting mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs—most notably under evasion—one might reasonably expect GNN defenses to remain effective in this paradigm. This expectation does not

hold. Unlike the traditional GNN setting, *graph-aware LLMs are additionally susceptible to feature perturbations, which renders these defenses ineffective*.

One prominent GNN defense makes the adjacency matrix trainable, letting edge weights adapt during training so as to counteract adversarial perturbations by assigning lower weights to edges connecting dissimilar nodes and higher weights to edges linking feature-similar nodes. Several **preprocessing-based** defenses build on this principle—for instance, removing edges between nodes of low feature similarity (Wu et al. 2019) or introducing randomized edge dropping (Dai et al. 2018). Critically, however, all such defenses depend on clean, unperturbed node features to compute reliable similarity scores. When the features are themselves perturbed, as in our attacks, these defenses not only fail but may actively exacerbate the problem by forming spurious edges. We illustrate this failure on a random node from the Cora dataset (Figure 8): whereas the clean feature-similarity graph preserves some original edges while introducing a few incorrect ones, the graph reconstructed from perturbed features preserves none of the original connections and yields an entirely erroneous reconstruction.

A second line of work seeks to decouple the model’s reliance on graph structure. Shanthamallu, Thiagarajan, and Spanias (2021), for instance, trained a surrogate predictor that depends solely on node features and used an uncertainty-matching strategy to distill graph information from the GNN, while Wu et al. (2021) introduced a co-training framework that trains submodels independently on the feature and structural views and performs knowledge distillation by admitting confidently predicted unlabeled data into the training set. Yet these approaches, too, presuppose access to unperturbed node features—an assumption violated in our setting, where both structure and features are adversarially perturbed. Consequently, structural defenses developed for GNNs do not transfer directly to the graph-aware LLM paradigm, which demands defenses tailored specifically to its dual vulnerability—exactly the gap GALGUARD is designed to close.

Table 10: Illustration of the visually imperceptible homoglyph attack. The LLM processes the perturbed text as a distinct character sequence, yet its rendered form is nearly indistinguishable from the original to a human reader, as the image shows.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgprir huristic fr discvring prtln squnc pttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f stttisticl mdl to grup of rltd sauncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the visually imperceptible reordering attack. The LLM processes the perturbed text as a reordered character sequence, yet its rendered form is nearly indistinguishable from the original to a human reader, as the image shows.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplclpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

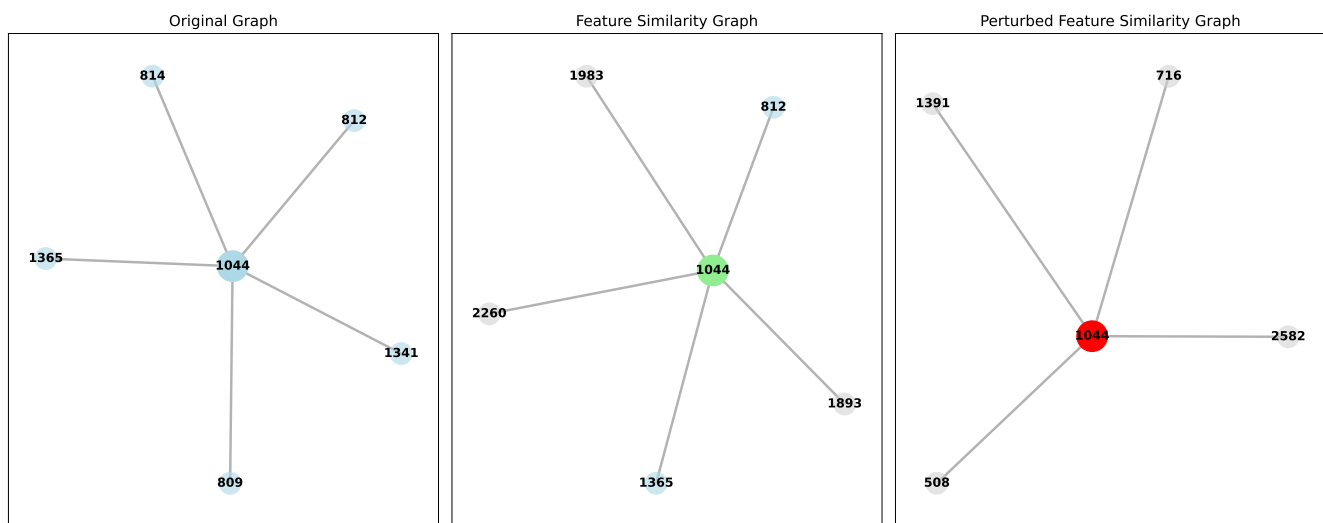


Figure 8: Edges reconstructed for a random node (id 1044). **Original (left):** the ground-truth edges from the original graph. **Feature Similarity (middle):** edges reconstructed from clean node features, with blue nodes marking connections preserved from the original graph. **Perturbed Feature Similarity (right):** edges reconstructed from perturbed node features, preserving none of the original connections. The contrast makes explicit why similarity-based defenses collapse once the features themselves are perturbed.